Introduction to ROS - Homework 1

Prof. Markus Ryll

April 17, 2026

1 Exercises

1.1 Submission

To submit your solutions you are required to create a repository on GitLab (this will be the first part of this exercise and most likely you have done that already during class). In GitLab you can specify who can clone your project by setting the 'Project visibility'. Please make sure that your project is an 'internal project' such that I can clone it (under Setting/General).

I will clone all repositories from GitLab on Monday, 21th of April end of day. This will be considered as your submission.

1.2 Exercise Part 1: Git

1. Create a repository (if not done already)
2. Add me as reporter member
3. Copy the ssh-link to your repository and student ID to the following google docs file: `https://docs.google.com/document/d/1gc-13NiWG1qIv4dd1BEGCvxG-QhJh6lwa9oLVlAx3wY/edit?usp=sharing`

1.3 Exercise Part 2: C++

The goal of this exercise is to refresh your C++ skills and learn how to compile and run a program under Linux. In particular, you will practice separating code into multiple files using header files and classes.

Setup First, install the required compiler tools:

```
sudo apt-get install build-essential
```

Preparation Clone the course repository:

```
git clone https://gitlab.lrz.de/aas_public/introduction-to-ros-2026.git
```

Either copy the folder `exercise_1` into your own repository or fork the repository.

Task Inside the folder `exercise_1`, you will find the files `calculator.cc` and `calculator.h`. Your task is to restructure and extend this program.

- Create a new file `main.cc`.
- Move the function `main()` from `calculator.cc` into `main.cc`.
- Move the declaration of the class `MyMath` into `calculator.h`.
- Ensure that `main.cc` includes `calculator.h`.

- In `main()`, create an instance of the class `MyMath`.
- Implement the following member functions of `MyMath`:
  - `double sum(double a, double b);`
  - `double square(double a);`
  - `double subtract(double a, double b);`
  - `double divide(double a, double b);`
  - `double sum(const std::vector<double>& values);`
  - `double surface(double radius);`
  - `std::vector<double> crossProduct(const std::vector<double>& a, const std::vector<double>& b);`
  - `std::vector<std::vector<double>> matrixMultiply(const std::vector<std::vector<double>>& A, const std::vector<std::vector<double>>& B);`
- The function `sum` should be overloaded such that one version sums two numbers and another version sums all elements of a vector.
- The function `surface` should compute the surface area of a circle.
- The function `crossProduct` should compute the vector product of two 3-dimensional vectors.
- The function `matrixMultiply` should compute the multiplication of two 2×2 matrices.
- Use your `MyMath` object in `main()` to test the implemented functions and print the results.

Compilation Compile and run your program using:

```
g++ main.cc calculator.cc -o calculator
./calculator
```

Expected Output Your program should print correct results for the implemented test cases.

Bonus (optional) Handle division by zero and invalid vector or matrix dimensions gracefully.

Submission Push your solution to your repository.